

Proceso de Desarrollo de Software

Trabajo Práctico Integrador – Sokoban

Profesor:

Jonathan Leonel Pepe

Los Javafantásticos – Integrantes:

LU 1196974 – CARRIAGA, Valentín

LU 1196402 – LENCINA, Dario Javier

LU 1192688 – MARTORELLA, Franco

LU 1209127 – PACHECO, Thomas

LU 1207775 – PERNA, Michelle Jazmín

UADE

Índice

Introducción y contexto	3
Arquitectura general	3
Modelo de dominio y polimorfismo	4
Patrones de diseño aplicados (GoF)	4
Adapter	4
Memento	4
Observer	4
Factory	5
Strategy	5
State	5
Template Method	5
Singleton	5
Principios SOLID y GRASP	5
SOLID	5
GRASP	6
Requisitos funcionales implementados	6
Funcionalidades adicionales	7
1. Repetición de la partida (replay)	7
2. Power-ups (ítems de piso)	7
Decisiones de diseño destacadas	8
Repositorio en GitHub de la aplicación	8
Diagrama de clases UML	8
Video de demostración (Gameplay)	9
Declaración del uso de inteligencia artificial	9
Conclusión	9

Introducción y contexto

Este informe documenta las decisiones de diseño y su justificación en la implementación del clásico juego de lógica Sokoban, desarrollado como Trabajo Integrador de la asignatura Proceso de Desarrollo de Software. El objetivo central del trabajo no es únicamente que el juego funcione, sino aplicar de forma correcta y justificada los principios y patrones de diseño vistos en la cursada: SOLID, GRASP, la arquitectura MVC y los patrones de diseño GoF (Gang of Four).

Sokoban consiste en empujar cajas dentro de un almacén hasta ubicarlas sobre sus casilleros de destino, completando cada nivel de la manera más eficiente posible. El proyecto está desarrollado íntegramente en Java, utilizando Swing para la interfaz gráfica 2D, y los niveles se cargan desde archivos de texto que describen la disposición inicial del tablero mediante caracteres.

A lo largo del documento se describe la arquitectura general, el modelo de dominio, los patrones y principios aplicados con su justificación, los requisitos funcionales cubiertos, las dos funcionalidades adicionales incorporadas y, finalmente, la declaración del uso de herramientas de inteligencia artificial.

Arquitectura general

La solución se organiza siguiendo el patrón arquitectónico **Modelo-Vista-Controlador (MVC)**, con una separación estricta de responsabilidades reflejada en la estructura de paquetes:

- **Model:** el estado y las reglas del juego (Tablero, la jerarquía de ElementoBase, cajas, terrenos, muros, ítems, jugador, las fábricas y el memento). No conoce a Swing ni a la vista.
- **View:** todo lo relacionado con la representación gráfica en Swing (Ventana, Menu, VistaJuego, TableroPanel, HUDPanel y las pantallas de fin de partida, victoria, paso de nivel y repetición).
- **Controller:** la orquestación de casos de uso (GameController y GameFlowController), la gestión de niveles (GestorNiveles), la entrada por teclado (InputController y MovimientoTeclado), el historial de deshacer y el puntaje (GestorDePartida y Caretaker), la grabación/reproducción de la partida (Grabacion y ReproductorPartida) y el sistema de sonido (GestorSonido).
- **Interfaces:** las abstracciones que desacoplan las capas (ISuscriptor, IMovimientos, ILectorNiveles).

El flujo de una acción es: la entrada del usuario llega al controlador, éste delega la regla al modelo, el modelo modifica su estado y notifica a la vista, y la vista se redibuja. Esta dirección de dependencias (la vista y el controlador conocen al modelo, pero el modelo no conoce a ninguna clase concreta de la vista) es la clave para mantener el modelo reutilizable y testeable.

Modelo de dominio y polimorfismo

El corazón del modelo es la clase abstracta **ElementoBase**, de la que heredan todos los elementos del tablero (paredes, suelos, aceite, destinos, cerrojos, rejas, las distintas cajas, los ítems y el jugador). Cada elemento responde de manera polimórfica a un conjunto de consultas de comportamiento: si bloquea el paso, si es empujable, si es resbaladizo, si es ocupable, qué ocurre cuando el jugador lo pisa y qué ocurre cuando una caja es empujada.

Este diseño es deliberado: el Tablero delega la lógica de movimiento en **MotorFisico**, que opera una sola vez en términos abstractos preguntándole a cada celda por su comportamiento a través de la interfaz **ITableroFisico**, en lugar de consultar su tipo concreto con condicionales. Gracias a esto, agregar un nuevo tipo de elemento (por ejemplo, un ítem nuevo o un tipo de caja) no requiere modificar el algoritmo de movimiento.

El tablero se representa con **dos grillas superpuestas**: una grilla inferior con el terreno fijo (suelo, aceite, destinos, cerrojos, rejas e ítems) y una grilla superior con las entidades móviles (el jugador y las cajas). Esta separación simplifica enormemente el movimiento y el dibujado: mover una caja implica actualizar sólo la capa superior, mientras que el terreno de abajo permanece intacto, permitiendo saber siempre qué había debajo de una caja o del jugador.

Patrones de diseño aplicados (GoF)

Adapter

El sistema de efectos de sonido se implementa con el patrón **Adapter**. La interfaz **IReproductorSonido** define el contrato de reproducción de audio, y **GestorSonido** adapta la API de `javax.sound.sampled.Clip` a esa interfaz. El **GameController** depende únicamente de la abstracción, permitiendo intercambiar la implementación (por ejemplo, por una implementación silenciosa para pruebas) sin modificar la lógica del juego. **GestorSonido** **combina** **Adapter** con **Singleton**, ya que gestiona una única instancia con todos los clips precargados durante la partida.

Memento

El requisito de deshacer movimientos se resolvió con el patrón **Memento**. La clase **TableroMemento** captura una instantánea completa del estado del tablero (posición y orientación del jugador, posición y resistencia de cada caja, ambas grillas, el estado de las rejas y el elemento bajo el jugador) sin exponer la representación interna del Tablero.

El Tablero actúa como **originator** (crearMemento/restaurar) y el **Caretaker** administra el historial. La consigna de deshacer (restaurar cinco movimientos hacia atrás, hasta tres usos consecutivos, sobre una ventana de quince movimientos) se implementa acotando la pila de mementos del **Caretaker**.

Observer

La comunicación entre el modelo y la vista utiliza el patrón **Observer** a través de la interfaz **ISuscriptor**. El **TableroPanel** se suscribe al Tablero y, tras cada cambio de estado, el modelo lo notifica para que se redibuje. Es importante destacar que el Tablero depende únicamente de la abstracción **ISuscriptor** y no de la clase concreta de la vista, lo que respeta la inversión de dependencias. Adicionalmente, para eventos puntuales (cambio de estado para el puntaje, fin de partida y efecto de pisar una celda) se emplean callbacks funcionales, una variante liviana del mismo principio de notificación.

Factory

La creación de objetos se centraliza en dos fábricas. **ElementoFactory** traduce cada carácter del archivo de nivel en el objeto de dominio correspondiente, y **TableroFactory** recorre la matriz de texto y ensambla el Tablero completo (grillas, lista de cajas, objetivos y jugador). Esto aísla la lógica de construcción del resto del sistema: cambiar cómo se crea un elemento, o agregar uno nuevo, se hace en un único lugar.

Strategy

La entrada del jugador se modela con el patrón **Strategy** mediante la interfaz **IMovimientos**. La implementación actual, **MovimientoTeclado**, traduce las teclas direccionales (o WASD) en acciones de movimiento. Al depender de la abstracción, el controlador podría incorporar otra estrategia de entrada (por ejemplo, un control por botones en pantalla o un jugador automático) sin modificar el resto del código.

State

El comportamiento de las **rejas** depende de su estado (abierta o cerrada): una reja cerrada bloquea el paso y no es ocupable, mientras que una abierta se comporta como suelo. Este cambio de comportamiento según el estado es una aplicación del patrón **State**, implementado mediante la interfaz **IEstadoReja** con dos implementaciones concretas: **RejaCerrada** (bloquea el paso) y **RejaAbierta** (se comporta como suelo). Las rejas se abren automáticamente cuando una caja llave se coloca sobre su cerrojo correspondiente.

Template Method

Los ítems de piso comparten un esqueleto de comportamiento común implementado con el patrón **Template Method**. La clase abstracta **ItemPiso** define el método **aplicar()**, que ejecuta un paso variable (**aplicarEfecto**) y luego siempre reemplaza el ítem por suelo (el ítem se consume al pisarlo). Cada ítem concreto (moneda, bomba, undo extra) sólo implementa su efecto particular, sin repetir la lógica de consumo.

Singleton

Se aplica **Singleton** en aquellos recursos de los que debe existir una única instancia durante toda la partida: la Ventana principal (para no romper el **CardLayout** ni el ciclo de vida de **Swing**), el **GameController** (para asegurar un único punto de orquestación), el **GestorSonido** (para centralizar la reproducción de audio), el **GestorNiveles** (para mantener consistente el progreso del jugador) y las fábricas (que son sin estado y se comparten). Las pantallas individuales también se manejan como instancias únicas dentro del **CardLayout** de la ventana.

Principios SOLID y GRASP

SOLID

- **Responsabilidad Única (SRP)**: cada clase tiene un motivo de cambio bien definido: el Tablero mantiene el estado del nivel y consulta las reglas (**ReglasDelJuego**), el **GameController** orquesta, las vistas dibujan, las fábricas construyen y el **Caretaker** administra el historial.
- **Abierto/Cerrado (OCP)**: gracias al polimorfismo de **ElementoBase**, el sistema está abierto a nuevos elementos (cajas, ítems, terrenos) pero cerrado a la modificación del algoritmo de movimiento de **MotorFisico**.

- **Sustitución de Liskov (LSP):** cualquier subclase de ElementoBase, Caja o ItemPiso puede usarse donde se espera el tipo base sin alterar la corrección del programa.
- **Segregación de Interfaces (ISP):** las interfaces son pequeñas y específicas. En particular, **IContextoItem** expone a los ítems sólo las operaciones que necesitan (sumar bonus, terminar la partida, sumar un undo extra) y no todo el controlador.
- **Inversión de Dependencias (DIP):** el modelo depende de abstracciones (ISuscriptor, IContextoItem) y el controlador de I lectorNiveles e IMovimientos, nunca de implementaciones concretas de capas superiores.

GRASP

- **Creator:** las fábricas son las responsables de crear los elementos y el tablero.
- **Controller:** GameController y GameFlowController concentran la coordinación de los casos de uso del juego.
- **Alta cohesión y bajo acoplamiento:** la separación MVC y el uso de interfaces mantienen a cada módulo enfocado y con dependencias mínimas.
- **Polimorfismo:** el comportamiento de cada celda se resuelve por herencia y no con estructuras condicionales sobre el tipo.
- **Fabricación pura (Pure Fabrication):** clases como Caretaker, Grabacion o las fábricas no representan conceptos del dominio, sino que se crearon para mejorar el diseño.
- **Indirección y Variaciones Protegidas:** las interfaces (I lectorNiveles, ISuscriptor, IContextoItem) actúan como punto de indirección que protege al sistema frente a cambios; por ejemplo, la carga de niveles podría migrar de texto a otro formato sin afectar al resto.

Requisitos funcionales implementados

- **Tablero y elementos:** paredes, espacios, destinos, terreno resbaladizo (aceite), cerrojos y rejas, representados gráficamente con imágenes.
- **Tipos de caja:** normal, frágil (con resistencia que se reduce en cada empuje y se rompe al llegar a cero) y llave (que abre las rejas al ubicarse sobre un cerrojo).
- **Movimiento:** el jugador se desplaza con las flechas o WASD, sólo empuja una caja por vez, no puede tirar de ellas ni atravesar cajas, paredes o muros cerrados.
- **Terreno resbaladizo:** al empujar una caja sobre aceite, ésta se desliza hasta chocar con un obstáculo; el mismo efecto aplica al jugador.
- **Deshacer:** botón de undo que retrocede cinco movimientos por uso, hasta un máximo de tres usos.
- **Condición de victoria y avance de nivel:** cuando todas las cajas están sobre sus destinos, se completa el nivel y se avanza al siguiente.

- **Puntaje:** al finalizar cada nivel se muestra un resumen; el puntaje parte de una base por nivel y se penaliza según la cantidad de movimientos y de empujes, sumando los bonus obtenidos.
- **HUD e interfaz:** se muestran movimientos, empujes, nivel actual y estado del undo, con botones visibles para deshacer, reiniciar el nivel y volver al menú.
- **Niveles:** se cargan desde archivos de texto en caracteres a través de `ILectorNiveles`.
- **Sonidos:** se incorporaron efectos de sonido que acompañan cada acción del jugador: movimiento, empuje de cajas, recolección de ítems (moneda, bomba, undo extra), rotura de cajas frágiles, apertura de rejillas, deshacer movimientos, paso de nivel, victoria y fin de partida.

Funcionalidades adicionales

La consigna exige incorporar dos funcionalidades adicionales que impliquen la aplicación de patrones de diseño. El equipo desarrolló las siguientes:

1. Repetición de la partida (replay)

Se graba el cien por ciento de los movimientos de la partida y se permite reproducirla al finalizar el nivel. La funcionalidad **reutiliza el patrón Memento** ya presente para el undo: la clase `Grabacion` almacena la secuencia completa de instantáneas (que incluyen el memento junto con el conteo de pasos y empujes) sin descartar historial, a diferencia del `Caretaker`. El `ReproductorPartida` las restaura en orden sobre el mismo tablero a través de la interfaz `IReproductorVista`, redibujando cada cuadro.

La reproducción es automática con un temporizador y ofrece controles para pausar, avanzar y retroceder. Esta decisión demuestra la reutilización de un patrón existente para un nuevo caso de uso, sin duplicar la lógica de captura de estado.

2. Power-ups (ítems de piso)

Se incorporaron ítems que el jugador activa al pisarlos: la **moneda** otorga un bonus de puntaje, la **bomba** termina la partida y el **undo extra** concede un uso adicional del botón de deshacer. Todos heredan de `ItemPiso` y aplican el patrón **Template Method** (el esqueleto de "ejecutar efecto y consumirse" está en la clase base; cada ítem sólo define su efecto). Además, para no acoplar el modelo al controlador, los ítems se comunican con el juego a través de la interfaz `IContextoItem`, respetando la Segregación de Interfaces: un ítem sólo ve las operaciones que necesita.

Como complemento, se agregaron otras mejoras menores como la orientación del jugador (para mostrar el sprite mirando en la dirección del movimiento) y la pantalla de resumen entre niveles.

Decisiones de diseño destacadas

- **Doble grilla (terreno / entidades):** separar el terreno fijo de las entidades móviles simplifica el movimiento, el dibujado por capas y la restauración de estado, y evita perder información de lo que hay debajo de una caja o del jugador.
- **Memento reutilizado:** un mismo mecanismo de captura de estado sirve tanto para el undo como para la repetición de la partida, evitando duplicación.
- **ITableroFisico como abstracción del tablero:** la interfaz ITableroFisico expone únicamente las operaciones que MotorFisico necesita para mover elementos (consultar, asignar y limpiar celdas), desacoplando completamente la física del juego de la implementación concreta del Tablero.
- **IContextoItem como frontera:** interponer una interfaz estrecha entre los ítems del modelo y el controlador impide que el dominio dependa de detalles de la capa de control.
- **Desacople modelo-vista vía ISuscriptor:** el modelo notifica a través de una abstracción, de modo que no depende de ninguna clase concreta de Swing.
- **Callbacks para eventos de dominio:** los eventos puntuales (fin de partida, cambio de puntaje, pisar una celda) se propagan con callbacks, manteniendo al modelo agnóstico respecto de quién los consume.

Repositorio en GitHub de la aplicación

El repositorio de la aplicación está hosteado en GitHub. La dirección del mismo es:

<https://github.com/michellejazmin/sokoban>

Dentro del repositorio, el archivo **README.md** contiene instrucciones para correr el juego.

Diagrama de clases UML

Dentro del repositorio, se incluyen dos archivos:

- **sokoban.plantuml:** archivo para abrir con la aplicación PlantUML.
- **DiagramaDeClases.png:** el mismo diagrama, exportado en formato PNG para mayor comodidad.

Video de demostración (Gameplay)

Video que demuestra las características del juego:

<https://www.youtube.com/watch?v=plTDtLvrL6M>

Declaración del uso de inteligencia artificial

En conformidad con lo indicado en la consigna, se declara que durante el desarrollo se utilizaron las herramientas de inteligencia artificial Claude y OpenCode.

El uso de las mismas incluyó: discusión y validación de decisiones de diseño y selección de patrones, asistencia en parte de la implementación y en refactorizaciones orientadas a SOLID y a los patrones GoF y apoyo en la detección y corrección de errores.

El equipo de desarrollo comprende, valida y adapta la totalidad del código entregado y las decisiones de diseño adoptadas, y está en condiciones de defenderlas.

Conclusión

La implementación de Sokoban permitió aplicar de forma concreta y justificada los principios y patrones de la asignatura. La arquitectura MVC y el uso de interfaces mantienen las capas desacopladas; el polimorfismo y las fábricas dan flexibilidad para extender el juego; y patrones como Memento, Observer, Strategy, State, Template Method y Adapter resuelven necesidades reales del dominio en lugar de aplicarse de manera forzada.